

Nonlinear optimization problem

$$\min f(\vec{x}) \quad (\text{unconstrained})$$

without information about derivatives

- Nelder-Mead downhill simplex method
- Hooke-Jeeves method
- Simulated annealing
- tabu search
- genetic algorithms

first derivative

- gradient descent

$$\vec{x}_{k+1} = \vec{x}_k - c_k \nabla f(\vec{x}_k)$$

- quasi-Newton-descent

$$\vec{x}_{k+1} = \vec{x}_k + \vec{d}_k$$

$$\text{where } H_k \vec{d}_k = -\nabla f(\vec{x}_k)$$

(H_k approx. Hessian of $f: \nabla^2 f(\vec{x}_k)$)

second derivative

- Newton descent

$$\vec{x}_{k+1} = \vec{x}_k + \vec{d}_k$$

$$\text{where } H_f(\vec{x}_k) \vec{d}_k = -\nabla f(\vec{x}_k)$$

- Trust region method

- approx. f by Taylor polynomial of 2nd order

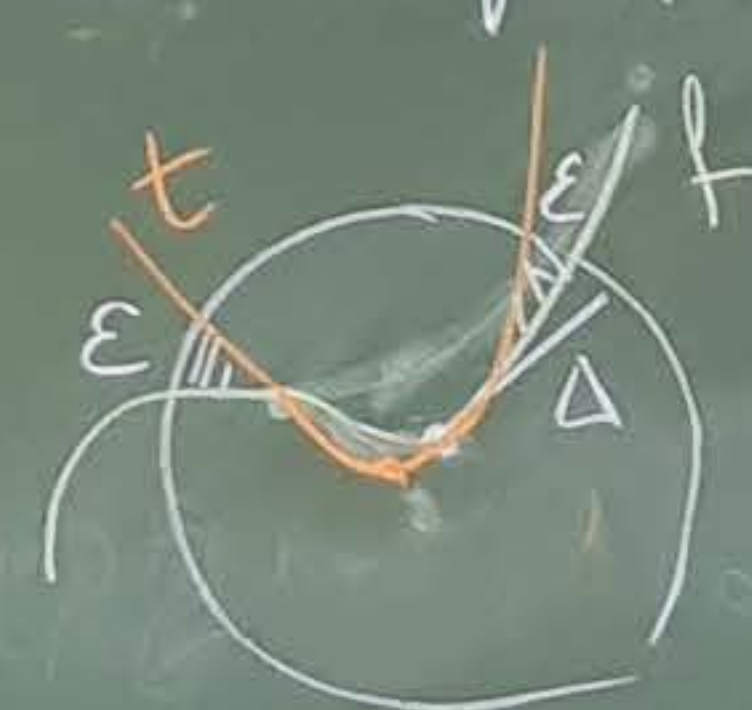
check accuracy of approx.

and adapt trust region

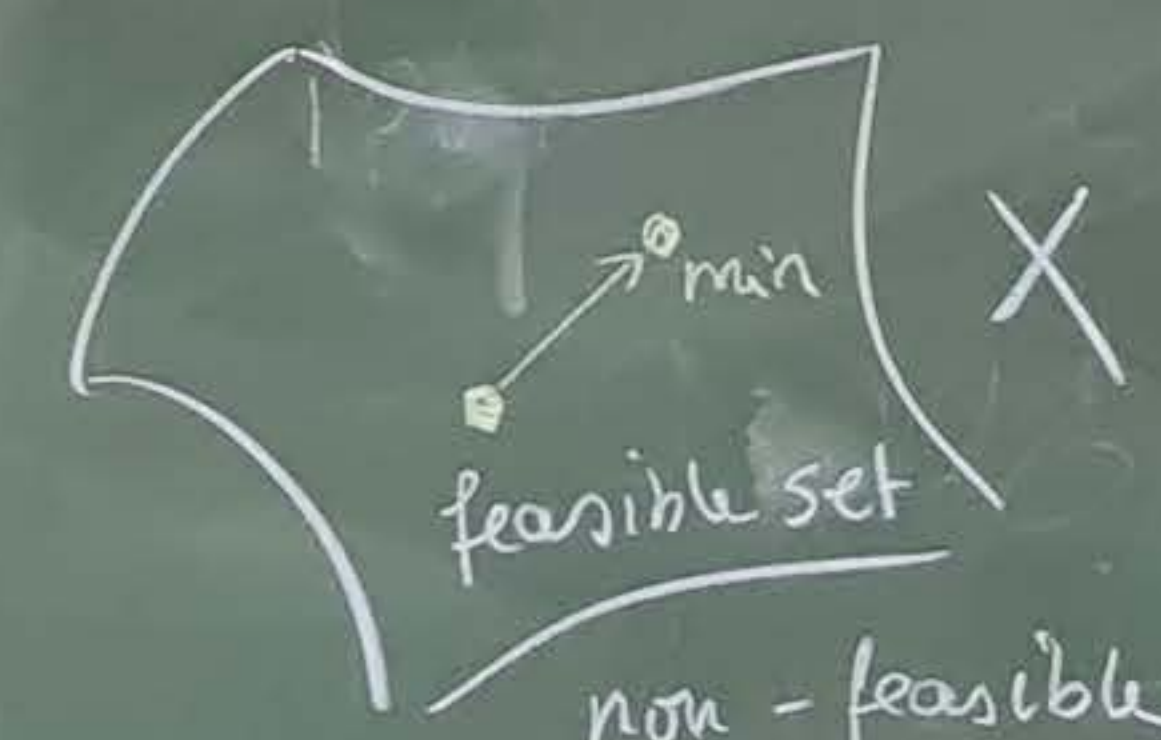
$$\min f(\vec{x}) \quad (\text{constrained})$$

where $\vec{x} \in X$

Trust region w. constraints



interior-point-methods



prevent crossing borders by barrier functions

$$X = \{ \vec{x} \in \mathbb{R}^n \mid h(\vec{x}) = \vec{0} \wedge g(\vec{x}) \leq \vec{0} \}$$

$$\min f(\vec{x}) + \sum \alpha \ln(-g_j(\vec{x})) + \sum \beta |h_k|$$

if $g_j(\vec{x}) \rightarrow 0 \rightarrow \ln(-g_j(\vec{x})) \rightarrow \infty$

Solvers for nonlinear optimization in Python

- `scipy.optimize.minimize` for both unconstrained and constrained problems

methods: unconstrained

Nelder-Mead

CG

BFGS

→ trust-neg

trust-exact

Newton-CG

constrained

→ trust-constr

SLSQP

conjugate gradient descent

quasi-Newton method

trust region combined w. CG

trust region

Newton combined w. CG

trust region w. constraints

and interior point alg

sequential quadratic programming approx.

- Pyomo: Ipopt interior point solver